REAL-TIME OPEN-SOURCE TRANSACTION MONITORING

Data Integration Software Development Kit

Product Guidance Document

Authors: **Justus Ortlepp - Tazama**

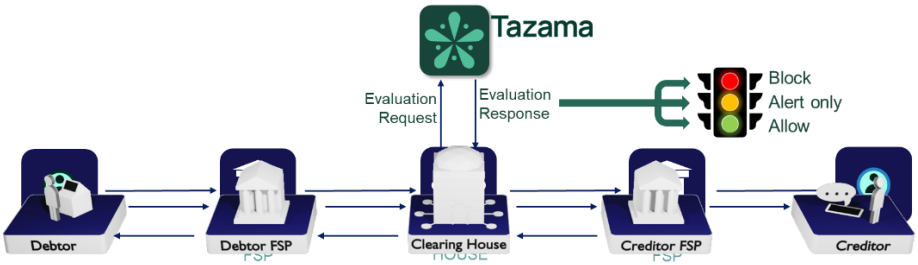
May 2025

Table of Contents

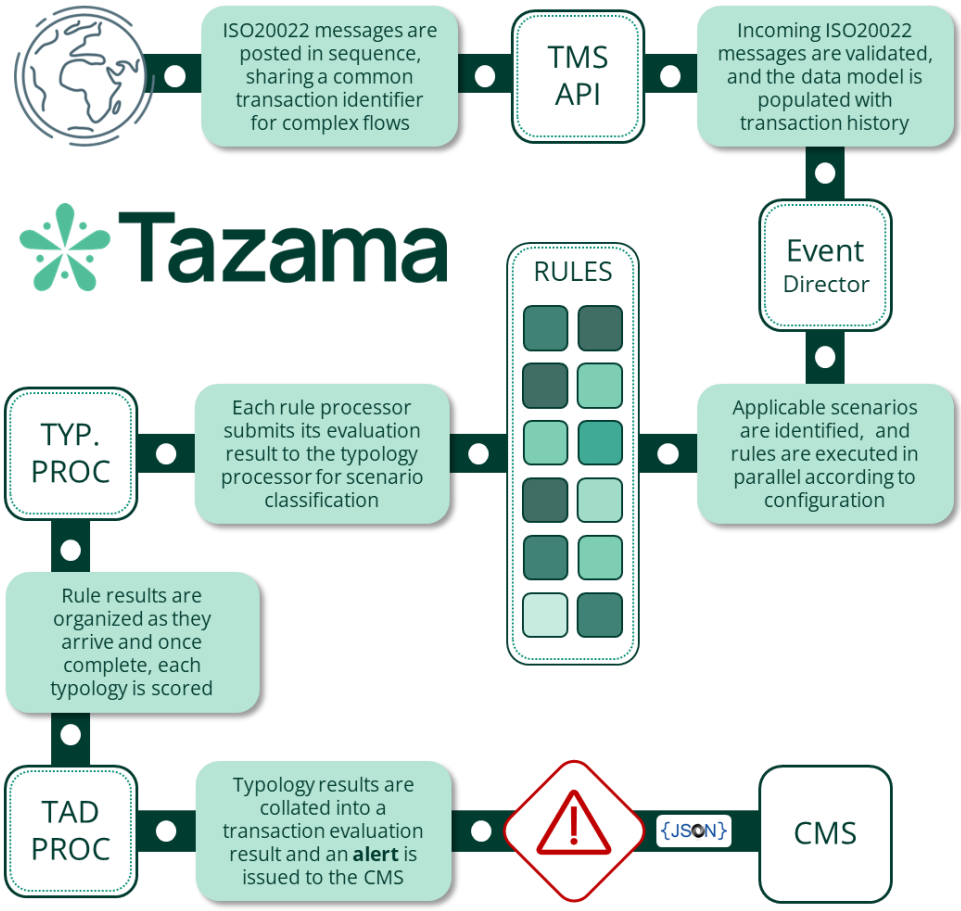
1 Introduction

1.1 Tazama System Overview

The Tazama Transaction Monitoring System is a rules-based forward-chaining inference engine that ingests transaction data as JSON-formatted ISO20022 messages and evaluates the data in real-time. Tazama uses a number of pre-built and configured rule processors to look for fraud and money-laundering behaviors. Rule results are summarized into fraud and money-laundering scenarios (known as typologies) and the system is able to block or flag transactions if typologies breach configured thresholds.



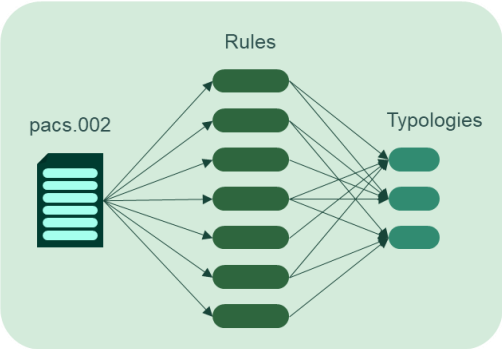
The core detection capability within the platform is distributed across 3 distinct steps in the end-to-end evaluation flow.



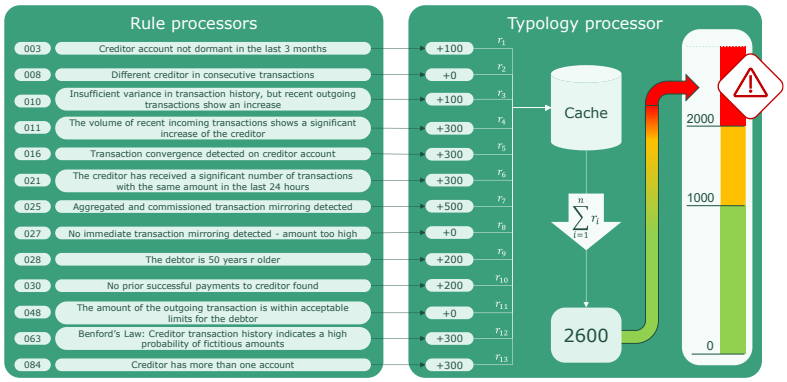
Once data is ingested into the transaction history by the TMS API, the Event Director (ED) performs an initial "triage" step to determine if the transaction should be inspected by the platform, and in what way. Currently this is a very simple decision based on the transaction type only (i.e. pain.001,

pain.013, pacs.008 and pacs.002), though we envisage that the decision-making here can be more complex in the future by inspecting attributes contained in the message. For now, the ED uses the transaction type to select the typologies that are to be evaluated and triggers the rules required by the typologies. The ED routing is configured via a network map that defines the hierarchy of typologies and rules.

Each rule processor that receives the trigger payload from the Event Director evaluates the transaction and the historical behavior of the transaction participants according to the rule's specification and configuration. Rule processors are driven by a combination of parameters and result category specifications to determine only one of a configurable set of related outcomes. The rule outcome is then submitted to the typology processor for scoring.



The typology processor assigns a weighting to each rule outcome as it is received based on the rule's parent typologies' configurations. Once all the rule results for a specific typology have been received, the typology adds all the weighted scores together to form the typology score. The typology score can be evaluated against an "interdiction" threshold to determine if the client system should be instructed to block a transaction "in flight". The typology configuration also contains an investigation threshold that will trigger a review or investigation process at the end of the transaction evaluation.

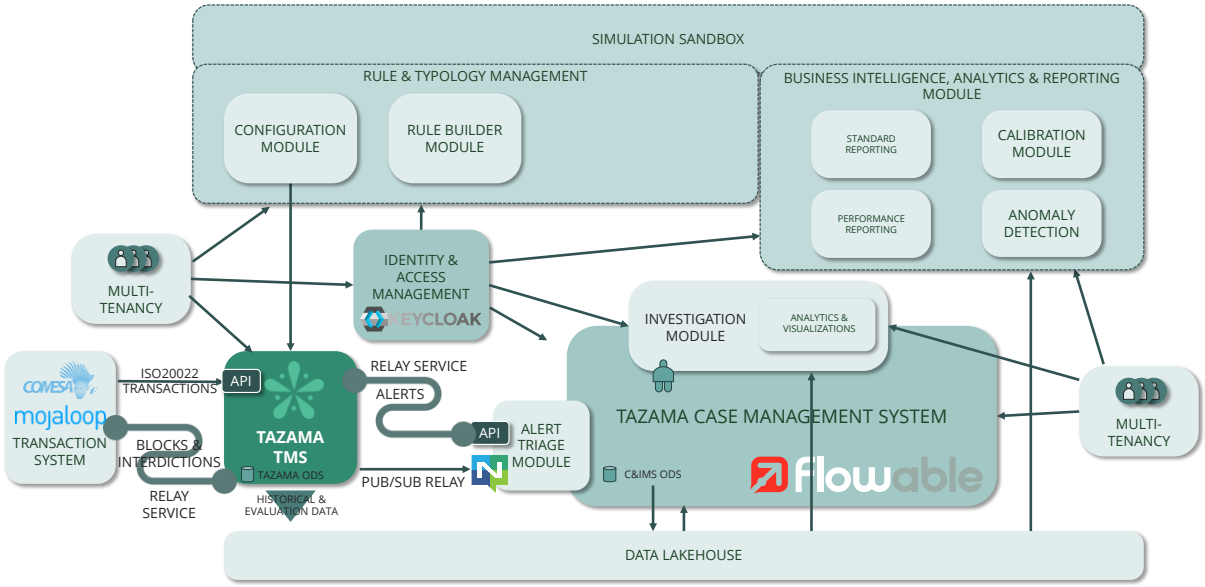


Once these three steps are complete, the evaluation of the transaction is wrapped up in the Transaction Aggregation and Decisioning Processor where the results from typologies are combined and reviewed to determine if an investigation alert should be sent to the Case Management System. If any typology had breached either its investigation or interdiction threshold, the system would trigger an investigation alert for that transaction.

The Tazama system is currently provided as a software engine and as such is generally implemented as a back-end service that taps into transaction flows.

1.2 Tazama Ecosystem Overview

The Tazama Transaction Monitoring Service is intended as a core component of a wider ecosystem that comprises of several components designed to combine into a comprehensive fraud and money-laundering management ecosystem.



These features can be summarized as follows:

1.2.1 Data Integration Software Development Kit

This component is designed to accelerate the deployment of Tazama into a new environment by allowing an implementer to rapidly create and implement REST API-based interfaces and end-to-end data pipelines in the Tazama Transaction Monitoring Service.

1.2.2 Relay Service Enhancement

The Relay Service component in the Tazama ecosystem is a light-weight point-to-point router for message traffic generated by Tazama into downstream systems, such as the Case Management System and upstream systems, such as the client transaction system. The Relay Service is designed to be highly modular and extensible and allows for multiple output interfaces that can be selected at deployment, with an easy-to-use integration pattern for adding additional target interfaces.

1.2.3 Case & Investigation Management System

Every alert that is generated out of the Tazama Transaction Monitoring Service (TMS) must be investigated to either confirm or refute the suspicious behavior identified by the TMS. This is firstly required to maintain the good health of the monitoring service itself, but if suspicious behavior is confirmed, the operator of the TMS also has an obligation to investigate the alert for fraud and money-laundering regulatory and legislative enforcement purposes.

The Case and Investigation Management System provides the processes and the tools to assist investigators in the fulfilment of this obligation.

The Case and Investigation Management System also contains an AI-powered Alert Triage Module that tries to automatically validate, classify and prioritize the alerts generated out of the TMS before an investigator is assigned to minimize unnecessary investigative effort.

1.2.4 Data Lakehouse

The Tazama Data Lakehouse ingests evaluation results data and transaction history as an output of the TMS, and also provides a centralized analytics storage solution for further data enrichment and analysis in support of case management and model development and refinement. Tazama has implemented a data lakehouse, as opposed to a more traditional data warehouse or data lake, to provide the most appropriate foundation in support of AI and Machine Learning processing.

1.2.5 Business Intelligence, Analytics and Reporting

To unlock the most value out of the data that Tazama accumulates as part of its core transaction monitoring function, the Tazama ecosystem contains a fully integrated and flexible Business Intelligence and Analytics platform built around JupyterLab, one of the foremost tools for data science, Artificial Intelligence, and Machine Learning available as an Open-Source Software product today.

The Tazama Business Intelligence, Analytics and Reporting component provides the reporting and advanced modeling capabilities for model management and model development, as well as standard reporting for overall operational management of the ecosystem and regulatory reporting to fulfil regulatory obligations.

1.2.6 Model Management

The Tazama Model Management component, containing a configuration utility and a low-code rule-builder, provides accessible tools to an operator to add new rules and typologies and fine-tune their existing rules and typologies.

1.2.7 Simulation Sandbox

The Business Intelligence, Analytics and Reporting module, and the Model Management module, are supported by a simulation sandbox where new models can be tested before they are deployed into a production system.

1.3 Document Purpose

The purpose of this document is to provide an overview of the vision for the Data Integration Software Development Kit to aid in the design and development of this component in the Tazama ecosystem.

2 Data Integration Software Development Kit Overview

2.1 Problem Statement

Data integration is one of the most labor-intensive and time-consuming activities in any system deployment project, especially for a system like Tazama where its effective functioning is directly influenced by its ability to ingest data from a wide variety of data sources. And given the wide variety of data sources and standards available, including standardized messages formats such as ISO20022 and ISO8583, and bespoke or proprietary message formats that are only discovered during an implementation, integration easily consume most of the project's technical capacity.

Compound this task with a possible lack of technical capability in a DIY setting, or a small system integrator's limited scale of operations, it is essential for this process to be automated as much as possible, and to be made as easy as possible, for Tazama to be accessible to all users.

2.2 The Proposed Solution

The Data Integration Software Development Kit (DISDK) provides a tool, or a series of tools, to assist in the rapid integration and deployment of the Tazama Transaction Monitoring Service.

The DISDK will allow users to easily create an interface for a new data message format, and easily thread this new data-flow through the system end-to-end to allow the user to create and implement rules to evaluate the new message, and then generate monitoring results according to the implemented rules.

In addition, the DISDK will allow for the ingestion of additional data that can be used to enrich or extend the data used for transaction monitoring.

We expect that there will be a trade-off between the accessibility offered by a dynamic interface as opposed to the performance offered by a static interface, but the strategic goal here is accessibility and being able to accelerate the adoption of Tazama, especially amongst smaller organizations. We expect that a larger organization with greater requirements for performance would also command the kind of resources (budget, skills) required for the development of static interfaces.

The modularized extensibility of the DISDK creates an additional opportunity for an organization to create "interface packs" as modular extensions to Tazama's monitoring scaffold and then to offer these as commercial products.

2.3 Conceptual Design

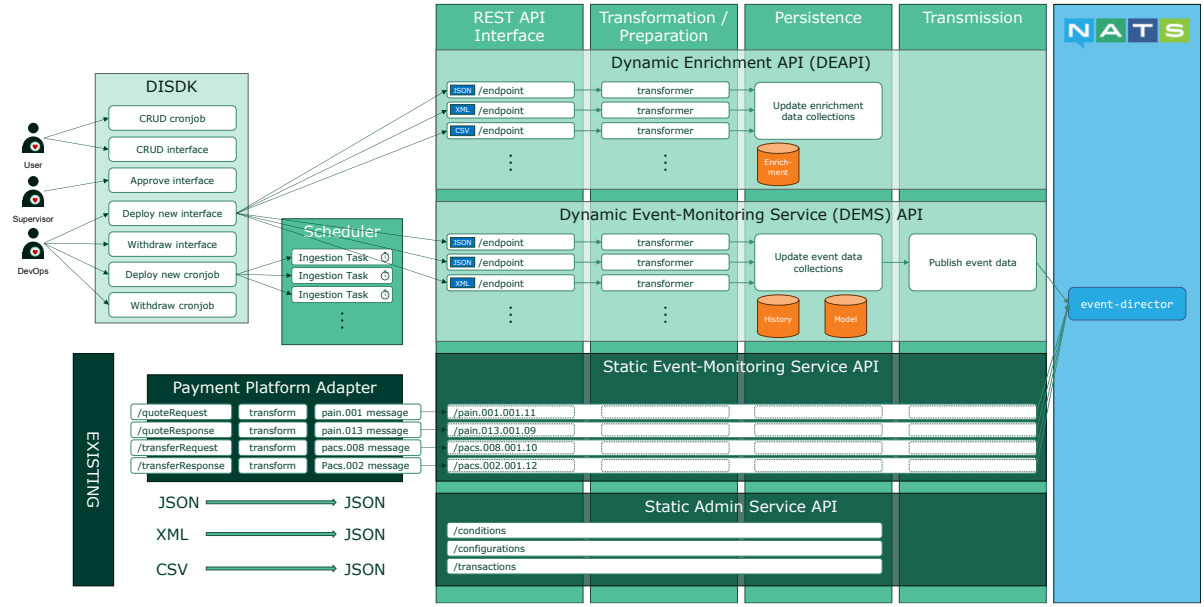


Figure: *DISDK scope and context*

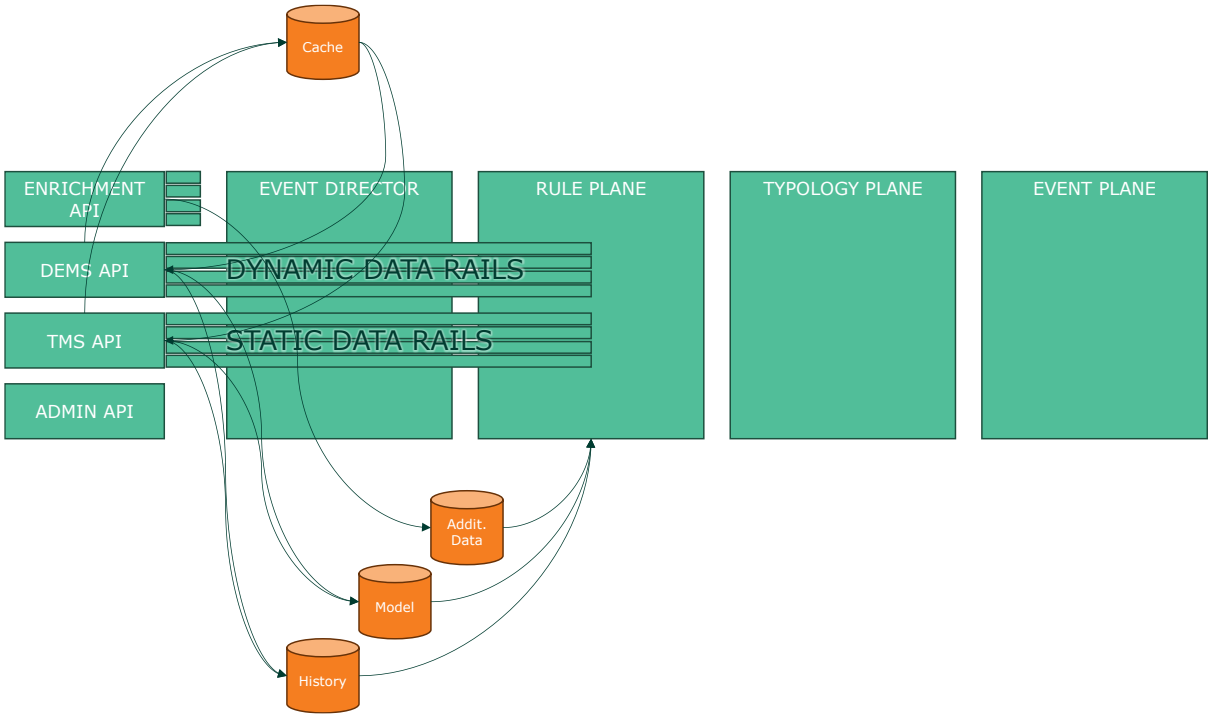


Figure: *Data rail scope across Tazama processes*

2.3.1 Components

2.3.1.1 Overview

Tazama currently only ingests data for Transaction Monitoring purposes, but with the implementation of the DISDK as an easy way to extend data ingestion scope and capabilities, Tazama can be extended to ingest data from other types of business process events such as customer onboarding processes, or customer platform interactions (password changes, balance enquiries), etc. This additional event data can be ingested in real-time and is intended to be integrated with the Tazama data model so that an organization can also develop and attach rules and behavioral typologies to these events.

Tazama will also have the capability to ingest data for enrichment purposes. This is data that can be added to the system for lookup purposes during the evaluation process to save Tazama from calling out to external systems during event monitoring. This ingestion mechanism will be activated manually in an ad hoc fashion, or automatically in a scheduled manner.

2.3.1.2 Data Integration SDK (DISDK)

The Data Integration SDK (DISDK) will allow users to create and maintain dynamic interfaces into the core Tazama Transaction Monitoring System. In general, there should be 3 different classes of users to create (user), approve (supervisor), and deploy (devops) the various interface components required for the implementation of a new interface.

With the DISDK, operators will be able to deploy JSON and XML interfaces directly in a new Tazama Dynamic Event Monitoring Service API for real-time ingestion from a wide variety of organization data streams.

2.3.1.3 The Current (Static) Transaction Monitoring Service (TMS) API

2.3.1.3.1 Core Functionality

The TMS service handles four types of ISO 20022 financial transaction messages:

- Pain001 - Customer Credit Transfer Initiation
- Pain013 - Creditor Payment Activation Request
- Pacs008 - Financial Institution to Financial Institution Customer Credit Transfer
- Pacs002 - Payment Status Report

2.3.1.3.2 Architecture

The service follows a modern microservice architecture:

- **API Layer**: Fastify server for handling HTTP requests
- **Processing Logic**: Core business logic in logic.service.ts
- **Caching**: Valkey for distributed caching
- **Database**: ArangoDB for transaction history and relationships
- **Messaging**: NATS for event-driven communication with other services
- **Monitoring**: Elastic APM for performance and error tracking
- **Containerization**: Docker for deployment

2.3.1.3.3 Main Components

Entry Point (`index.ts`)

The `index.ts` file initializes the service:

1. Sets up configuration from environment variables
2. Initializes the database connection
3. Connects to NATS messaging system
4. Starts the Fastify HTTP server
5. Implements clustering for better performance using Node.js cluster module

Controllers (`app.controller.ts`)

The `app.controller.ts` file defines request handlers for the four message types:

1. Each handler extracts the transaction type from the request URL
2. Validates the request body against JSON schemas
3. Delegates processing to the corresponding handler in the logic service
4. Returns appropriate HTTP responses

Business Logic (`logic.service.ts`)

The `logic.service.ts` contains the core processing logic:

1. Extracts relevant data from transaction messages
2. Creates relationships between transaction entities
3. Caches transaction data for faster retrieval
4. Implements transaction processing workflows
5. Manages data persistence in the database
6. Key functions:
   a. Message handler functions `handlePain001`, `handlePain013`, `handlePacs008`, `handlePacs002` for processing each message type
   b. `rebuildCache` for cache reconstruction if needed
   c. `parseDataCache` utility for extracting data for caching

Each of the four handlers (`handlePain001`, `handlePain013`, `handlePacs008`, `handlePacs002`) follows these standardized steps:

1. Initialization
   a. Log start of handling process
   b. Create APM (Application Performance Monitoring) span
   c. Record start time for performance tracking
   d. Extract transaction ID
2. Message Type Tagging
   a. Set transaction type (`TxTp`) from input parameter
   b. Attach type to transaction object
3. Data Extraction
   a. Parse relevant fields from the transaction object
   b. Extract identifiers (`MsgId`, `EndToEndId`, `PmtInfId`)
   c. Extract financial data (Amount, Currency, Exchange Rate)
   d. Extract timestamps (`CreDtTm`)

4. ID Construction
    a. Build composite IDs for accounts and entities
    b. Format: `{ID}{SchemeCode}{MemberId}`
5. Relationship Creation
    a. Construct a `TransactionRelationship` object
    b. Define source/destination (from/to)
    c. Attach transaction metadata
6. Cache Construction
    a. Create a standardized `DataCache` object
    b. Populate with normalized data
    c. Attach to the transaction object
7. Database Operations
    a. Use `Promise.all` for concurrent operations where possible
    b. Store transaction history
    c. Create/update account records
    d. Create/update entity records
    e. Create relationship records
8. Error Handling
    a. Try/catch blocks around database operations
    b. Standardized error logging
    c. APM span ending in finally blocks
9. Event Notification
    a. Send response to event-director service
    b. Include performance metrics
    c. Include trace parent for distributed tracing
10. Completion
    a. End APM span
    b. Log completion of handling process

Caching Layer

Uses ValKey for caching transaction data with configurable Time-to-Live (TTL) settings.

Database Management

Stores transaction relationships, entity data, and transaction history in ArangoDB.

Transaction Flow

1. HTTP request received with ISO 20022 message body
2. Request validated against schema
3. Message processed by corresponding handler based on type
4. Data extracted from message (IDs, amounts, account info, etc.)
5. Transaction decomposition (entity, account, and transaction information)
6. Transaction context created
7. Data persisted in database and cache
8. Transaction sent via NATS to Event Director service

2.3.1.3.4 Message Transmission

Messages are routed between Tazama components using a NATS publish/subscribe protocol. The TMS API follows these steps to send a message to the **`event-director`** NATS subject:

1. Handler Preparation (in `logic.service.ts`)
   a. Transaction object assembled with:
      i. Original ISO 20022 message data
      ii. Extracted and normalized `DataCache`
      iii. Performance and tracing metadata
2. Invocation of `StartupFactory` Method
   a. The message is passed to the `handleResponse` method
3. Message Processing in `StartupFactory` (Inside `handleResponse`)
   a. Topic Selection
      i. Constructs the NATS topic name using configuration
      ii. Typically follows pattern like `tms-service.response`
   b. Message Serialization
      i. Uses Protocol Buffers for efficient serialization
      ii. The `createMessageBuffer` function converts the JS object to a binary buffer
      iii. This ensures standardized message format across the system
   c. Connection Validation
      i. Checks if NATS connection exists and is active
      ii. If not connected, attempts to reconnect before sending
   d. Publish to NATS
      i. Publishes the serialized message to the NATS server
      ii. Uses the constructed topic name
      iii. Passes the binary buffer as message payload
   e. Delivery Confirmation (Optional)
      i. If configured, waits for confirmation that message was received
      ii. Uses NATS flush operation to ensure delivery
   f. Error Handling
      i. Catches and logs any errors during publishing
      ii. Rethrows to allow calling code to handle failures

2.3.1.4 The Dynamic Event Monitoring Service (DEMS) API

The DISDK must have the ability to create modular and deployable interface packages to a new dynamic REST API framework, the Dynamic Event Monitoring Service (DEMS) API, designed to host and interpret these packages at run-time. While the end-to-end process for handling each new dynamic endpoint will be similar to the way messages are handled in the TMS API, we can break the process for the DEMS API down into 4 discrete steps. These steps are expected to mirror the functionality of the TMS API, but in a way that allows the interface and its processes to be dynamically managed.

1. **Step 1: REST API interface processing**
   a. HTTP request received with message body
   b. Request validated against schema
2. **Step 2: Data Transformation and Preparation**
   a. Message processed by corresponding handler based on type

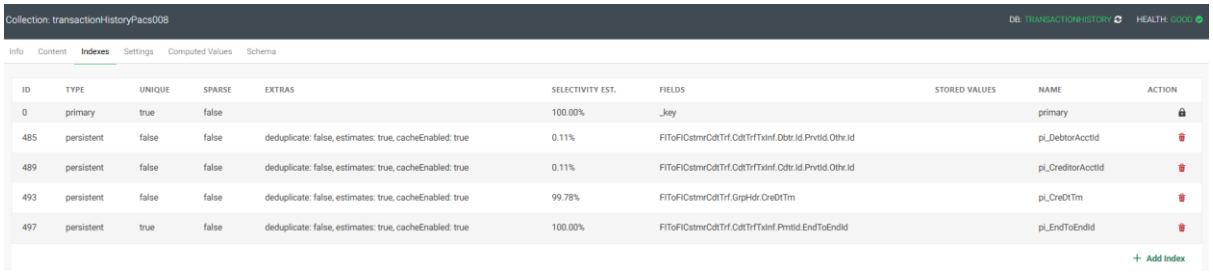
   b. Data extracted from message (IDs, amounts, account info, etc.)
   c. Transaction decomposition (entity, account, and transaction information)
   d. Transaction context created
3. **Step 3: Persistence**
   a. Data persisted in database and cache
4. **Step 4: Transmission**
   a. Transaction sent via NATS to Event Director service

3 The Tazama Databases

3.1 transactionHistory

The **`transactionHistory`** database will contain a separate database collection for each message received at the DEMS API in its original unadulterated format.

Fields that are used to optimize the retrieval of these messages are indexed, for example:



3.2 pseudonyms

The pseudonyms graph database contains node and edge collections to represent the components of the Tazama data model and their relationships to each other. These components are:

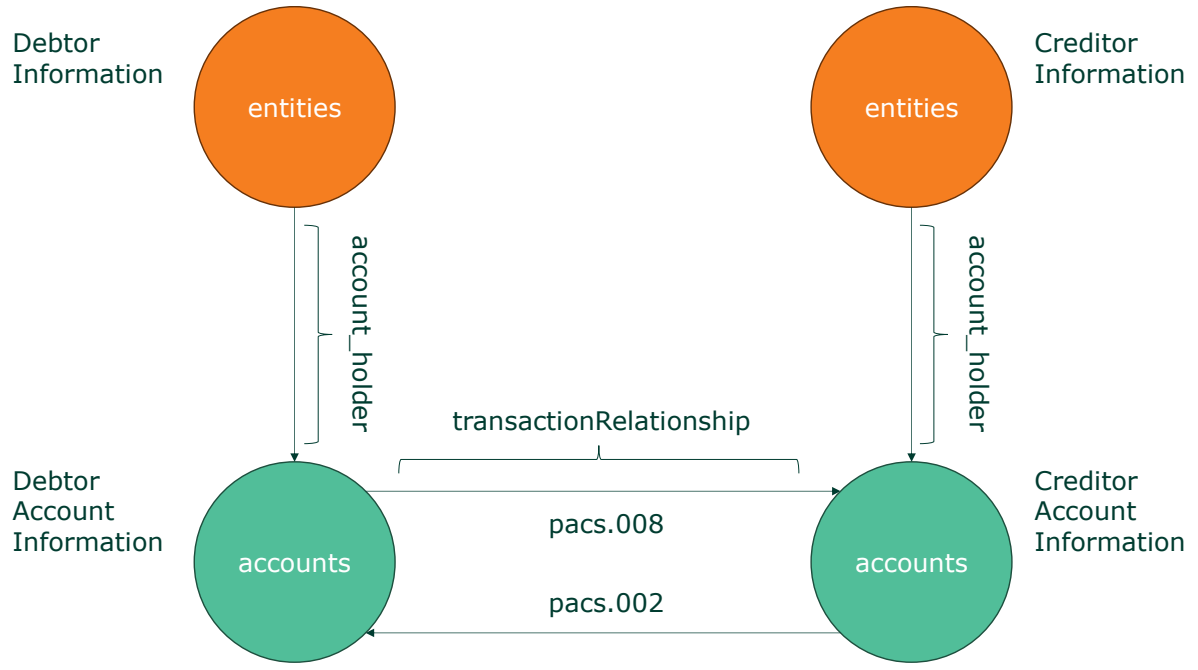
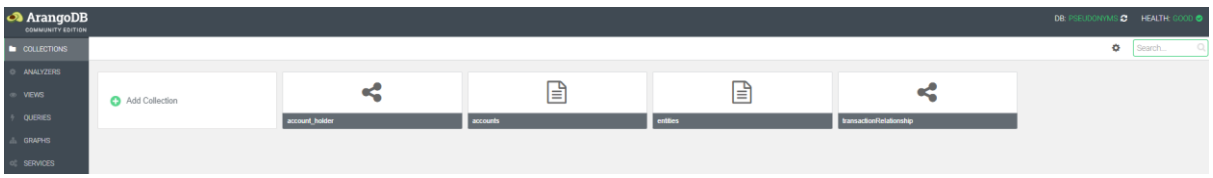
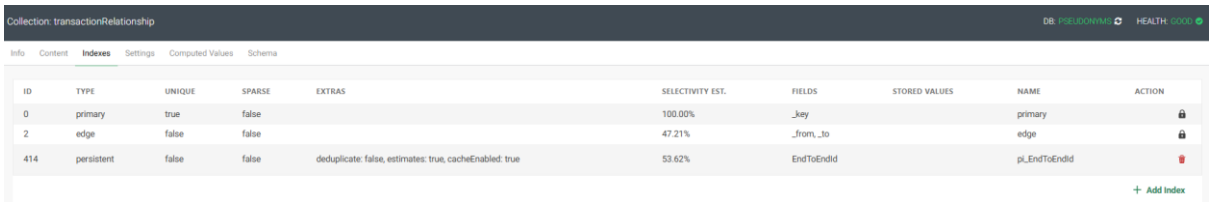


Figure: *Current Tazama data model*



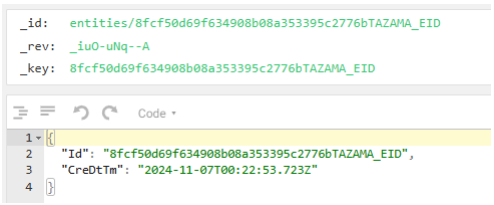
By default, all _keys, _to, and _from fields are indexed in a graph database, but additional fields used for optimized retrieval may also be indexed:



The contents of the data model is typically sparse and the data model only contains information that is actually used by rule processors.

These are the contents of the various data collections:

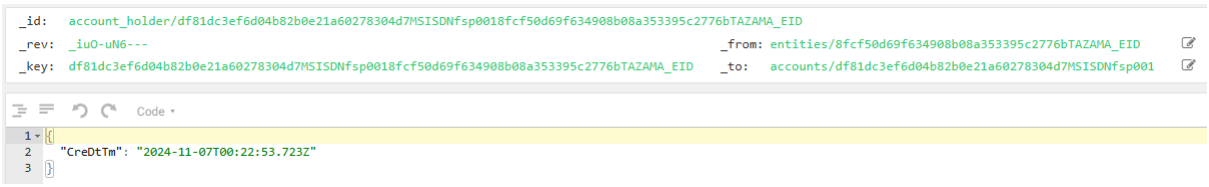
3.2.1 entities



Attribute	Description
_id	The **_id** field is composed out of the collection name (**entities**) followed by the **_key** value.
_key	The **_key** field is the unique identifier for the document (record) in the **entities** node collection. The key is a unique, predictable, and repeatable composite key generated by the TMS API out of the entity identifier and the identifier scheme name in the ISO20022 message's debtor or creditor object, e.g.: `{ "id": "8fcf50d69f634908b08a353395c2776b", "schmeNm": { "prtry": "TAZAMA_EID" } }` Tazama generates the key value from the incoming message to prevent the need to look it up when it is required. This key is used by Tazama to look up the entity document (record) directly. This field is indexed by ArangoDB by default.
_from	This collection is a node collection and does not contain the edge attributes.
_to	This collection is a node collection and does not contain the edge attributes.
Id	This is a copy of the **_key** value and is no longer used in Tazama - it is redundant. An index is created for this field at system deployment.
CreDtTm	The `CreDtTm` timestamp records the time when the entity record was created, for example when a transaction message is received for an entity

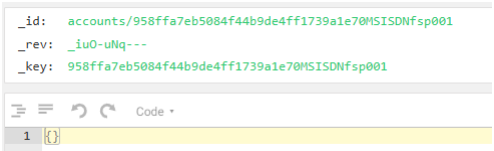
	for the first time or when a condition was created for an entity that had not yet transacted.

3.2.2 account_holder



Attribute	Description
_id	The **_id** field is composed out of the collection name (**account_holder**) followed by the _key value.
_key	The **_key** field is the unique identifier for the document (record) in the **account_holder** edge collection. The key is a unique, predictable, and repeatable composite key generated by the TMS API out of the entity identifier and the account identifier. Tazama generates the key value from the incoming message to prevent the need to look it up when it is required. This key is used by Tazama to look up the account document (record) directly. This field is indexed by ArangoDB by default.
_from	The **_from** field is the unique identifier for the entity document (record) in the **entities** collection. This key can be used by Tazama to look up all **account_holder** relationships for the entity directly. This field is indexed by ArangoDB by default.
_to	The **_to** field is the unique identifier for the account document (record) in the **accounts** collection. This key can be used by Tazama to look up all **account_holder** relationships for the account directly. This field is indexed by ArangoDB by default.
CreDtTm	The **CreDtTm** timestamp records the time when the account was linked to the entity, for example when a transaction message is received for an entity and account for the first time.

3.2.3 accounts



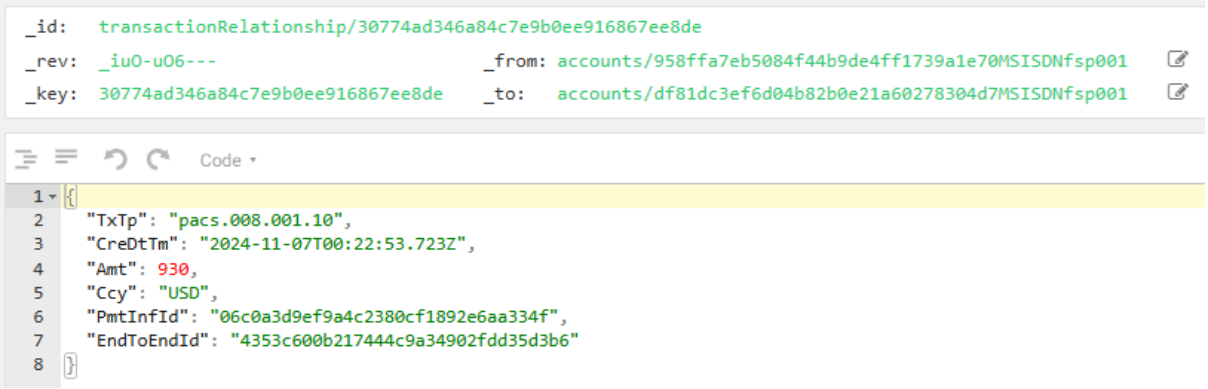
Attribute	Description
_id	The **_id** field is composed out of the collection name (**accounts**) followed by the **_key** value.
_key	The **_key** field is the unique identifier for the document (record) in the **accounts** node collection. The key is a unique, predictable, and repeatable composite key generated by the TMS API out of the entity identifier, the identifier scheme name, and the agent reference in the ISO20022 message's debtor account or creditor account object, e.g.:

	<pre>{
	  "id": " 5ff8010b6ecc4a6089b52d3a0d9bc41d",
	  "schmeNm": {
	  "prtry": "MSISDN"
	  },
	  "agt": {
	  "finInstnId": {
	  "clrSysMmbId": {
	  "mmbId": "dfsp001"
	  }
	  }
	  }
	}</pre>
	Tazama generates the key value from the incoming message to prevent the need to look it up when it is required. This key is used by Tazama to look up the entity document (record) directly. This field is indexed by ArangoDB by default.

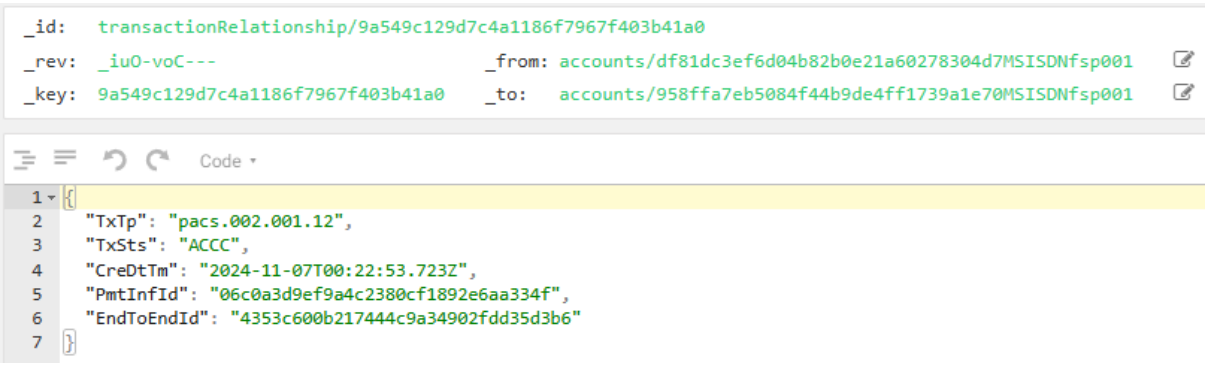
_from	This collection is a node collection and does not contain the edge attributes.
_to	This collection is a node collection and does not contain the edge attributes.
CreDtTm	The `CreDtTm` timestamp records the time when the account record was created, for example when a transaction message is received for an account for the first time or when a condition was created for an account that had not yet transacted.

3.2.4 transactionRelationship

pacs.008.001.10



pacs.002.001.12



Attribute	Description
_id	The **_id** field is composed out of the collection name (**transactionRelationship**) followed by the **_key** value.

_key	The **_key** field is the unique identifier for the document (record) in the **transactionRelationship** edge collection. The key is a unique and predictable key populated by the TMS API out of the message identifier in the ISO20022 message, e.g.:
	
	pacs.008.001.10
	
	`{ "FIToFICstmrCdtTrf": { "GrpHdr": { "MsgId": "30774ad346a84c7e9b0ee916867ee8de" } } }`
	
	pacs.002.001.12
	
	`{ "FIToFIPmtSts": { "GrpHdr": {`

	"MsgId": "9a549c129d7c4a1186f7967f403b41a0" } } } Tazama populates the key value from the incoming message to prevent the need to look it up when it is required. This key is used by Tazama to look up the entity document (record) directly. This field is indexed by ArangoDB by default.
_from[1]	The **_from** field is the unique identifier for the account document (record) in the **accounts** collection from where the transaction message originates (source). This key can be used by Tazama to directly look up all **transactionRelationship** messages that originated from the account. This field is indexed by ArangoDB by default. In the current implementation of ISO20022 in Tazama, the messages retrieved via the _from key would be pain.001 and pacs.008 messages sent by the debtor FI.
_to	The **_to** field is the unique identifier for the account document (record) in the **accounts** collection where the transaction message terminates (destination). This key can be used by Tazama to directly look up all **transactionRelationship** messages that flow to the account. This field is indexed by ArangoDB by default. In the current implementation of ISO20022 in Tazama, the messages retrieved would be pain.013 and pacs.002 response messages sent by the creditor FI.
Document contents	The contents of a particular message may vary depending on the information that is available in the message when it is received by Tazama and the information that was deemed necessary for effective rule execution. In general, we prefer data required for a rule to be added to the Tazama data model, with some very rare exceptions[2] for when data is required for only a single rule and the data can be looked up easily.

[1] A transaction message is mapped to an edge in the database between a source account in the **accounts** collection and a target account in the **accounts** collection, defined by who the debtor and creditor in a transaction is. Transactions flow from a debtor to a creditor; however, transaction messages may be transmitted from a debtor to creditor (such as a pacs.008 "Financial Institution (FI) to FI Credit Transfer" message to perform a transfer) or from a creditor to a debtor (such as a pacs.002 "FI to FI Payment Status" message that is sent in response to a specific pacs.008 message).

[2] Rule 007 retrieves the text description for a message from the transactionHistory database and rule 028 retrieves the debtor's date of birth from the transactionHistory database.

DISDK creates interfaces
DEMPS hosts them

4 Expectations

4.1 Event Monitoring

The expectations are that the DISDK will allow a user to create an entire end-to-end data rail for a new message, with specific attention to the following requirements for individual core Tazama Transaction Monitoring Service components:

4.1.1 TMS API

1. Avoid disturbing the existing (Static) TMS API by creating a new and separate Dynamic Event Monitoring Service (DEMS) API to host the interfaces created via the DISDK.

4.1.2 DEMS API

1. Create a new and separate Dynamic Event Monitoring Service (DEMS) API to host the interfaces created via the DISDK.
2. Allow a user to create new interfaces for real-time data ingestion as a deployable package via the DISDK.
3. The DISKD will be a "development" tool for use by developers and is not expected to be deployed into a Tazama "production" instance; however the tool will have its own independent development, test and production cycle.

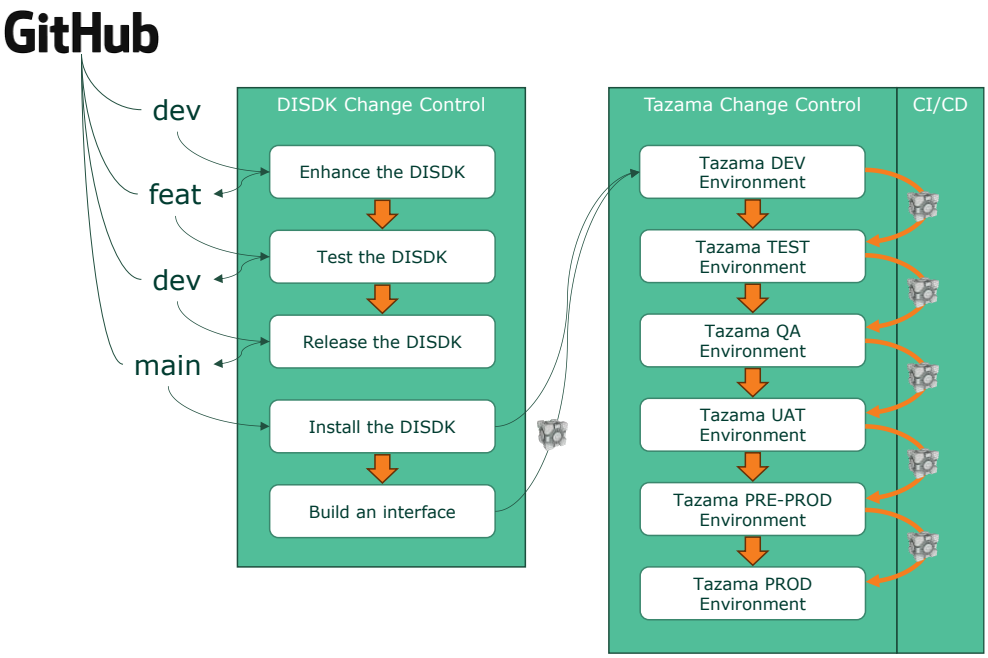


Figure: *DISDK vs Tazama change control*

a. The developers of the DISDK will build, enhance and maintain the DISDK following the Tazama change control process in GitHub (dev branch to feature branch to dev branch to main branch).
b. A developer will use a "production" version of the DISDK from the Tazama main branch in a Tazama "development" environment to build interfaces for a Tazama implementation.

c. The DISDK will not need any specific role-based access control for the developer to use the DISDK to build a new interface - it is a development tool.
d. A developer will build an interface package and deploy this package to a development instance of Tazama for unit testing.
e. If unit testing is successful, the package can be deployed to a test / QA / UAT / Pre-Prod environment, as required by the change control processes of the implementer, through an automated CI/CD process.
f. If acceptance testing is successful, the interface package can be deployed to the Tazama production environment's DEMS API through an automated CI/CD process. The CI/CD process will determine if the deployment to production will require system down-time or a hot-swappable update (e.g. traffic shifting/canary release, alias swapping, hot module replacement).
g. It is not an expectation that the DISDK project will deliver its own CI/CD process, but instead that the same CI/CD framework that governs the deployment of other Tazama components should be used.

4. Requirements for the endpoints that are created via the DISDK:

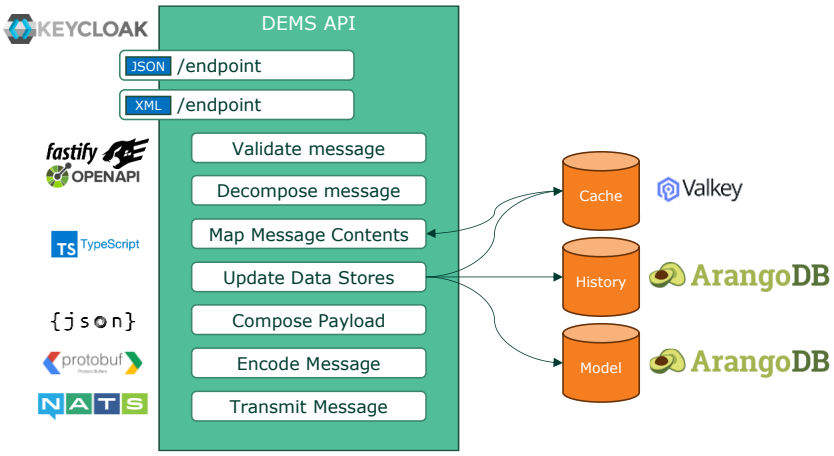


Figure: *DEMS API internal process steps and integration*

a. Restrict access to authorized users/systems only through authentication with KeyCloak via the Tazama authentication-service.
b. Users must be authenticated to use a specific endpoint against a claim set up for the endpoint in KeyCloak.
c. Messages received by an endpoint must be validated against a JSON or XML schema.
d. Messages received by an endpoint must be stored unadulterated (as received) in the Tazama `transactionHistory` database in a dedicated collection created for the specific message type.
e. Messages must be mapped or transformed to an internal Tazama JSON payload, the Tazama data model, and a cacheable object:

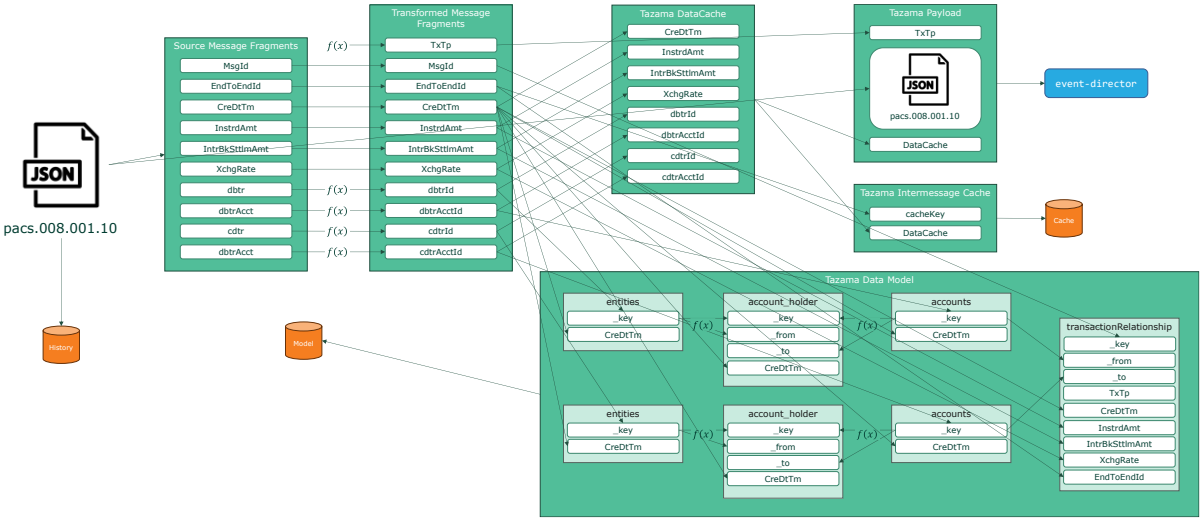


Figure: *pacs.008 example for dynamic interface handling*

i. A Tazama payload typically consists of a message type attribute (`TxTp`), the incoming message content, and a `DataCache` object.

ii. At the very least, a message received by the DEMS API (already in JSON format) may be stored as history, tagged with a message type, and then passed on to the Event Director with no further action required.

iii. In addition to the least steps, the following steps may be performed as specified by the DISDK:

   1. Translation of the message into a JSON object (not required if the message is already in a JSON format). All message-handling operations in Tazama are performed using JSON objects.
   2. Decomposition of the message into message fragments (either individual attributes, or entire objects), as required for integration into the data model, or for caching for integration into other messages. Message fragments will be used to assist in the transformation and the mapping of the JSON message to the Tazama data model, cached data, and the Tazama payload.
   3. Each message fragment can be used as is, or transformed in a variety of ways, including:
      a. Combination with other message fragments
      b. Combination with other static or dynamic information
      c. Transformed through a function of some kind (mathematical, text, etc.)
   4. New message fragments may be added:
      a. Combination with other message fragments
      b. Combination with other static or dynamic information
      c. Transformed from other message fragments through a function of some kind (mathematical, text, etc.)
      d. Defaulted with static or dynamic information
   5. If the message is to be integrated into the existing Tazama data model, the integration keys into points in the data model must be extracted from the message as fragments.
      a. For integration into, or addition to, the `entities` collection, the entity `_key`
      b. For integration into, or addition to, the `accounts` collection, the account `_key`

c. New collections may be created via the DISDK from time to time and published extensions to the data model must also be available in the DISDK for the integration of new message into the data model.

6. JSON objects across messages are sparsely populated, i.e. not all messages will contain the same information and none of the attributes of the content of any given data model object is expected to be mandatory. Only the keys required to connect a new message to a specific object in the data model are mandatory: if a key cannot be composed, the message cannot be connected.
7. Some fragments of the message may be used to populate a `DataCache` object. The `DataCache` object is a method for passing data between related messages as a shortcut to giving a later message access to an earlier message's data without forcing the later message to look up the earlier message in the database at great expense. A `DataCache` object may either be created out of a message and cached, or retrieved from cache and added to a message. In the cache, the `DataCache` object must be assigned a unique cache key by the earlier message that can be used by the later message to retrieve the `DataCache` object.

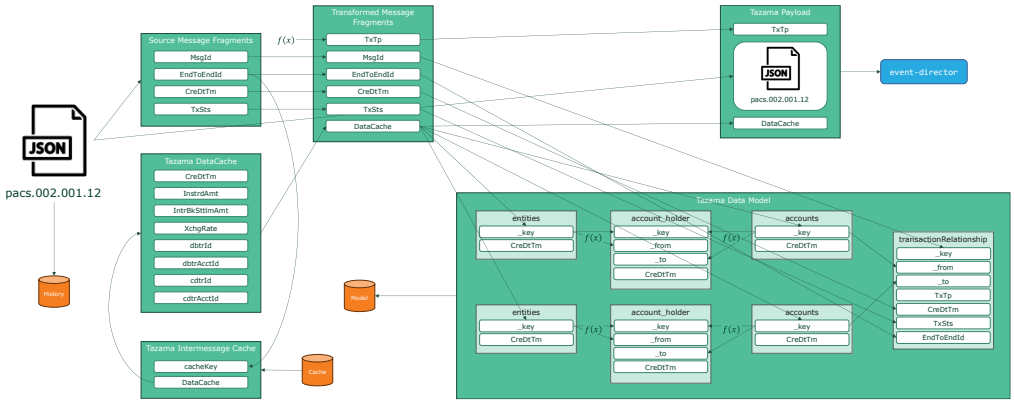


Figure: *pacs.002 example for cache retrieval*

iv. The Tazama logging framework must be implemented alongside the interface and log the processing of a new message according to the log-level configured for the DEMS API, from highly verbose TRACE logging to light production-level ERROR and WARN logging only.

v. Every message that is processed by the DEMS API must contain a `TxTp` attribute[3]. For convenience and consistency, the value of the `TxTp` attribute, the name of the API endpoint, and the name of the `transactionHistory` collection should all be the exact same text string, e.g. `pacs.008.001.10`.

5. Exceptions that require retrieving data for rule processing from the `transactionHistory` database must be entirely avoided. All data that is required for rule processing must be added to the Tazama data model in the `pseudonyms` database.
6. Tazama transmits messages between processors using a NATS publish/subscribe protocol. The message is encoded with Protocol Buffers for efficient serialization. The Tazama `frms-coe-lib`

[3] We would very much like to rename this to an `evtTp` (event type) attribute…

software library contains a Protocol Buffer Schema[4] as a universal container for all current message types (pain.001, pain.013, pacs.008, pacs.002). The Protocol Buffer Schema ensures type safety at serialization. The DISDK is intended to allow the creation of new interfaces in the DEMS API with variable structures which will not map to any of the existing message formats captured in the Protocol Buffer Schema. If users need to update the existing Protocol Buffer Schema when a new interface is added to the DEMS API, the entire Tazama deployment will have to be rebuilt end-to-end to adopt the change to the `frms-coe-lib` library. An alternative solution will be required that can still leverage Protocol Buffers as implemented, without requiring constant changes to the software library and redeployment of all system components.

4.1.3 Downstream processing

4.1.3.1 Event Director

The event director's `handleTransaction` function receives a deserialized message via NATS. This message is cast to type `any`[5] and relies on the type safety provided by the Procol Buffer Schema. The event director is able to inspect the payload as a normal JSON object and extracts the `TxTp` value from the message to route the transaction to the correct rule and typology processors.

This behavior is expected to prevail with the implementation of dynamic interfaces in the DEMS API. In the long term, the event director will also be required to route a message based on the values of other attributes in the message for more complex and fine-grained routing.

The event director's routing is configured through a network map that defines rules and typologies based on the `TxTp` value. The maintenance of the network map will be facilitated through the Config UI. All attributes in a message that is relevant to the routing, such as the `TxTp` field, will ultimately need to be communicated to the Config UI so that these attributes can be referenced in the construction of the network map. So, for example, if the DISDK adds a new interface for a new type of message, and that message contains new attributes that will inform the routing of the message inside Tazama, then the interface package that is created by the DISDK should also be available and readable to the Config UI so that the new attributes can be used in the development of the network map.

At the moment, the only useful attribute in the message is the `TxTp` field and at the very least, the Config UI should be aware of the different possible values of the `TxTp` field.

Only a pacs.002 message is currently routed for evaluation; all other messages are dropped by the event director when they are received and no route is identified for that message type in the network map.

4.1.3.2 The Rule Plane

The event director routes the message to all rules that are required to evaluate the message for suspicious behavior. All current rules are developed exclusively for the evaluation of a pacs.002 message. If a new message type is sent to a rule processor, the rule processor must be rewritten to interpret the new message type. Furthermore, the pacs.002 message context is understood by the

---

[4] https://github.com/tazama-lf/frms-coe-lib/blob/dev/src/helpers/proto/Full.proto
[5] https://github.com/tazama-lf/event-director/blob/d2a00955dd32962cb5728265c3d74d08040b1812/src/services/logic.service.ts#L47

rule processor and expressed in the query that is used to retrieve the transaction history, which means that the message context is effectively hard-coded in the rule processor. The rule processor knows that a pacs.002 message is a status message in response to a pacs.008 message, and in a Mojaloop environment, some rules even know that a pain.001 message is followed by a pain.013 message which is followed by a pacs.008 message and is finally followed by a pacs.002 message.

The implementation of the DISDK provides an opportunity for a user to define not only a variety of new messages, but also the composition and structure of the data model and the composition and structure of the message payload that will be evaluated. Messages in the DEMS API can be mapped to a common payload format that is more universal across different message types. At the moment, with the way the root object in an ISO20022 message is defined (e.g. `FIToFICstmrCdtTrf` for a pacs.008 message, or `FIToFIPmtSts` for a pacs.002 message) makes the messages impossible to handle in a consistent manner; however if the messages were mapped to a more generic payload, all rule processors will be able to interrogate the message contents consistently.

This mapping to a standard format still does not resolve the issue of message context, but it will make individual message handling simpler. Bespoke development of individual rule processors will be inevitable at some point. For that purpose, Tazama is proposing the development of a low-code/no-code Rule Builder that will allow a user to build rule processors more easily to allow for rules customization on a larger scale.

Like the Config UI, the Rule Builder will also need to be aware of the current messages, data model, and message payloads that have been implemented in the system through the DISDK. The interface package that is created by the DISDK should also be available and readable to the Rule Builder so that the new messages can be used in the development of new rules to evaluate them.

Note that a rule processor does have to decode the message's Protobuf stream in order to add its own output to the payload before re-encoding it again for transmission; however the solution that will be implemented to allow the transmission of dynamic messages using Protobuf should be transparent to a rule processor as well.

4.1.3.3 The Typology Plane

No specific impact related to the DEMS API is relevant to the typology processor at the moment. The typology processor does not interact with the message payload and only passes the payload on to the TADProc as part of its output exactly as it was received.

Note that the typology processor does have to decode the message's Protobuf stream in order to add its own output to the payload before re-encoding it again for transmission; however the solution that will be implemented to allow the transmission of dynamic messages using Protobuf should be transparent to the typology processor as well.

4.1.3.4 The Transaction / Event Plane

No specific impact related to the DEMS API is relevant to the transaction aggregation and decisioning processor (TADProc) at the moment. The TADProc does not interact with the message payload and only passes the payload on to the `evaluationResults` database as part of its output exactly as it was received.

Note that the TADProc does have to decode the message's Protobuf stream in order to add its own output to the payload before re-encoding it again for transmission; however the solution that will be

implemented to allow the transmission of dynamic messages using Protobuf should be transparent to the TADProc as well.

4.1.3.5 The Relay Services

No specific impact related to the DEMS API is relevant to the relay service at the moment. The relay service does not interact with the message payload and only passes the payload on to the integration destination as part of its output exactly as it was received.

Note that the relay service does sometimes have to decode the message's Protobuf stream and then translate the message to a raw JSON object before the message is relayed; however the solution that will be implemented to allow the transmission of dynamic messages using Protobuf should be transparent to the relay service as well.

4.1.3.6 Case and Investigation Management

An alert that is generated out of Tazama will contain the original message payload, as well as the detailed result of the evaluation. While the method of ingestion by the CMS has not been decided, the CMS must be able to cope with the expected variability in the structure of the messages submitted to Tazama via the DEMS API.

4.2 Dynamic Enrichment API

To a large extent the Dynamic Enrichment API (DEAPI) should provide for the same functionality as the DEMS API in terms of creating a dynamic endpoint, transformation, and database updates. Data ingested via the DEAPI is expected to be ingested in bulk (even if it is only one record), and not to be ingested for evaluation purposes. The destination of this data will be the database directly, and the intention is to be able to use this data to support the evaluation of messages that are ingested in real-time via either the DEMS or the TMS APIs.

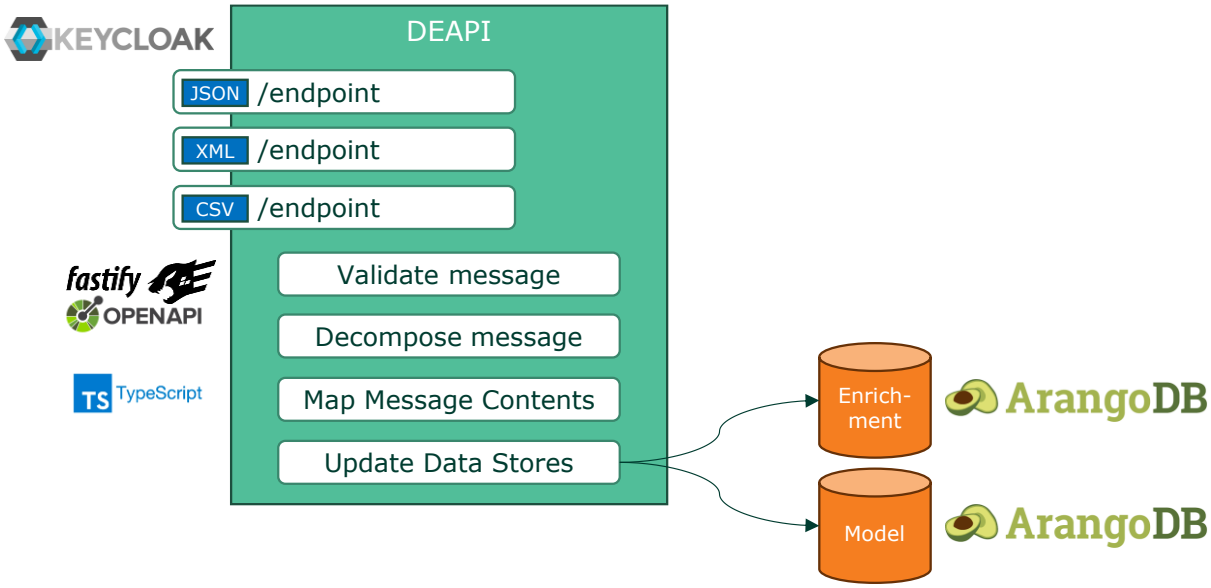


Figure: *Data Enrichment API internal process steps and integration*

1. The DEAPI must be able to ingest data in bulk, either as arrays of JSON objects, XML files with multiple root elements, or CSV files. Each element (record) of such a file will be expected to have an identical structure.
2. The DISDK must be allow a user to create an interface that can represent the individual records in the submitted data. The DEAPI must then allow the processing of the individual records against the interface specification in the same way that an individual record would be ingested through the DEMS API.
3. The DEAPI must also be able to update the Tazama data model in the same way that it would for a message ingested via the DEMS API, with the exception that this is intended to be a bulk process and that the messages will not be submitted for evaluation.
4. The DISDK should allow for the creation of the target database collection in the (new) `enrichment` database that will contain the data ingested via the DEAPI.
    a. The collection and the endpoint should have the sam0e name (e.g. `/unsanctions635` and `enrichment.unsanctions635`)
5. For future enhancement, the DISDK should also be able to populate an interface attached to the Tazama Data Lakehouse in the same bulk-supported way as the Tazama ODS is populated through the DEAPI.
6. For future enhancement, the DISDK should also be able to populate an interface attached to the Tazama Sandbox in the same bulk-supported way as the Tazama ODS is populated through the DEAPI.
7. While the development and maintenance of a DEAPI interface is intended to be a developer-driven process and expected to be facilitated through the implemented CI/CD for change control purposes, the scheduling of jobs to replicate data in bulk is an operational process. An authorized user must be able to configure a new scheduled replication in the specific environment for which the data is intended (dev, test, prod, etc.) through a user interface (UI), API, or Command-Line Interface (CLI).